



AIAA 3111 - Introduction to Data Mining

Lec-3: Pattern Mining: Basic Concepts and Methods

Dr. Zhidan Liu



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Lec 3-4: Two Lectures on **Pattern Mining**

Part 1 (80 minutes)	Part 2 (80 minutes)	Lecture 4
Transactions, support, rules Apriori and downward closure	ECLAT and FP-Growth Closed and maximal patterns Pattern evaluation	<i>Multi-level and multidimensional patterns Constraint-based, sequential, and subgraph mining</i>

**This week develops the common foundation.
Week 4 extends the pattern types.**

Learning Outcomes

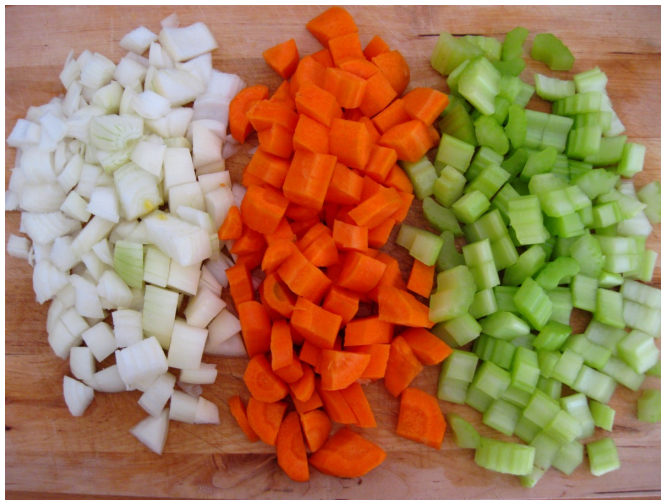
Represent	Compute	Trace	Evaluate
Convert records into transactions, items, itemsets, and rules	Calculate support, confidence, lift, and χ^2	Follow Apriori, ECLAT, and FP-Growth on small data	Choose a mining method and judge whether a rule is useful

You should be able to compute, trace, compare, and interpret.

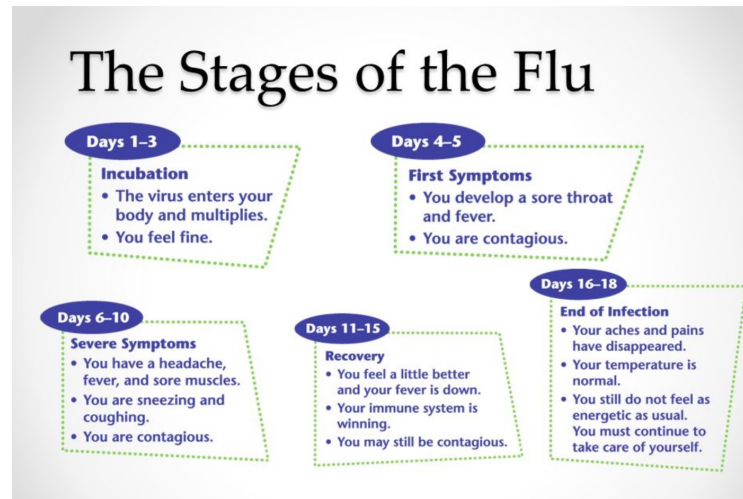
What Are Patterns?

❑ What are patterns?

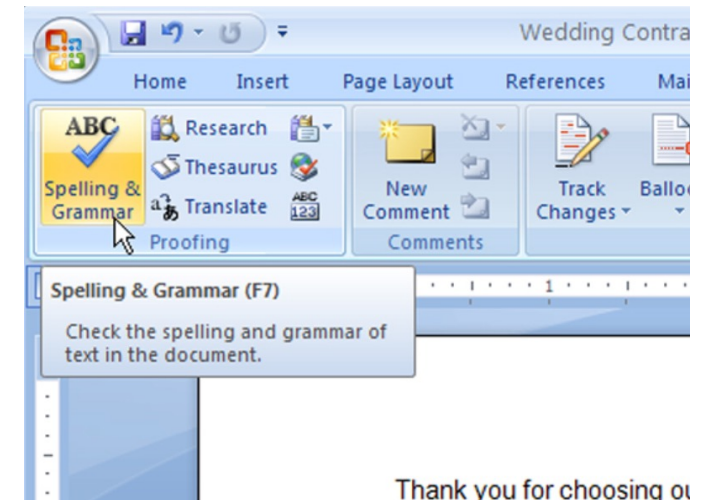
- ❑ **Patterns:** A set of items, subsequences, or substructures that *occur frequently together* (or *strongly correlated*) in a data set
- ❑ Patterns represent **intrinsic** and **important properties** of datasets



Frequent **itemset**



Frequent **sequences**



Frequent **structures**

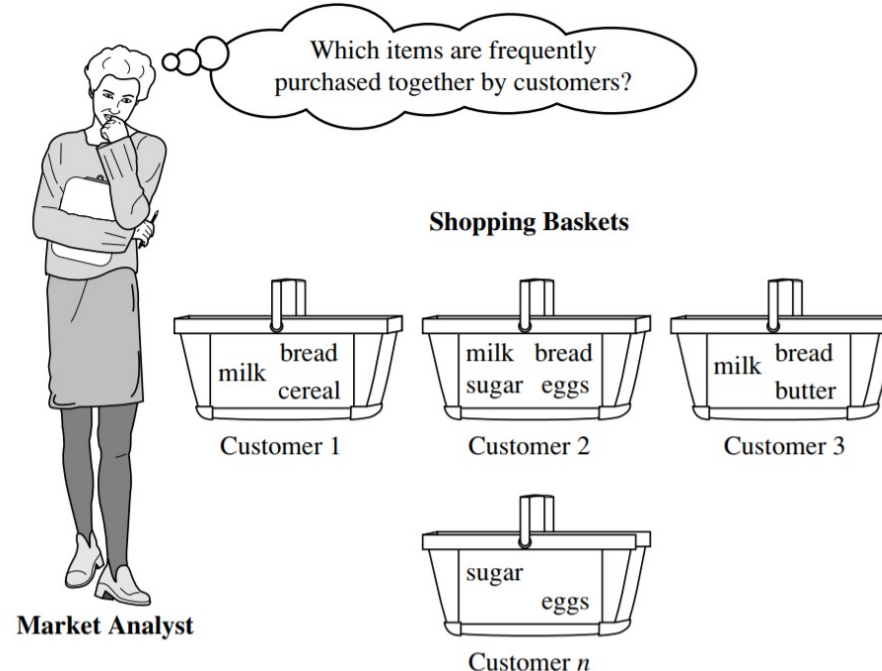
Pattern Types and Course Scope

Pattern form	Example	Coverage
Frequent itemset	Subscriber, Morning, Central	Week 3: full treatment
Association rule	Morning $\Rightarrow$ Subscriber	Week 3: full treatment
<i>Sequential pattern</i>	Search $\Rightarrow$ compare $\Rightarrow$ purchase	Week 4
<i>Structural pattern</i>	A recurring subgraph	Week 4
<i>Multi-level or constrained pattern</i>	Transport $\Rightarrow$ bike $\Rightarrow$ e-bike	Week 4

Itemsets provide the basic language for more advanced pattern types.

What Is Pattern Discovery?

- ❑ **Pattern discovery:** Uncovering patterns from massive data sets
- ❑ It can answer questions such as:
 - ❑ What products were often purchased together?
 - ❑ What are the subsequent purchases after buying an iPad?



Pattern Discovery: Why Is It Important?

- ❑ **Foundation** for many essential data mining tasks
 - ❑ Association, correlation, and causality analysis
 - ❑ Mining **sequential**, structural (e.g., sub-graph) patterns
 - ❑ **Classification**: Discriminative pattern-based analysis
 - ❑ **Cluster** analysis: Pattern-based subspace clustering
- ❑ Many types of data: spatiotemporal, multimedia, time-series, and stream
- ❑ Broad applications
 - ❑ Market basket analysis, cross-marketing, catalog design, sale campaign analysis, Web log analysis, biological sequence analysis

Basic Concepts: Transactional Database

- Transactional Database (TDB)
 - Each transaction is associated with an identifier, called a TID.
 - May also have counts associated with each item sold

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Basic Concepts: **k-Itemsets** and Their **Supports**

- Itemset: A set of one or more items

$$I = \{I_1, I_2, \dots, I_m\}$$

- k-itemset: An itemset containing *k* items:

$$X = \{x_1, \dots, x_k\}$$

Ex. {Beer, Nuts, Diaper} is a 3-itemset

- Absolute support (count)**

$\text{sup}\{X\}$ = occurrences of an itemset *X*

Ex. $\text{sup}\{\text{Beer}\} = 3$

Ex. $\text{sup}\{\text{Diaper}\} = 4$

Ex. $\text{sup}\{\text{Beer}, \text{Diaper}\} = 3$

Ex. $\text{sup}\{\text{Beer}, \text{Eggs}\} = 1$

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Relative support

$s\{X\}$ = The *fraction* of transactions that contains *X* (i.e., the probability that a transaction contains *X*)

Ex. $s\{\text{Beer}\} = 3/5 = 60\%$

Ex. $s\{\text{Diaper}\} = 4/5 = 80\%$

Ex. $s\{\text{Beer}, \text{Eggs}\} = 1/5 = 20\%$

Choosing the Transaction Unit

Transaction unit	Items can mean	Question answered
One trip	member type, time period, station, duration class	Which trip contexts co-occur?
One user-day	stations visited, categories used	Which activities occur on the same day?
One station-hour	weather, demand level, event status	Which operating conditions co-occur?

Note: Quantities such as trip counts can remain quantitative or be converted into items such as HighDemand.

Changing the transaction unit changes the meaning of support.

Shared-Bike Records as a Mining Example

trip_id	member	period	start station	duration
T1	Subscriber	Morning	Central	Short
T2	Subscriber	Morning	Central	Short
T3	Subscriber	Morning	Riverside	Short
T4	Casual	Weekend	Riverside	Long
T5	Casual	Weekend	Museum	Long
T6	Casual	Weekend	Riverside	Long
T7	Subscriber	Evening	Central	Short
T8	Casual	Weekend	Museum	Long

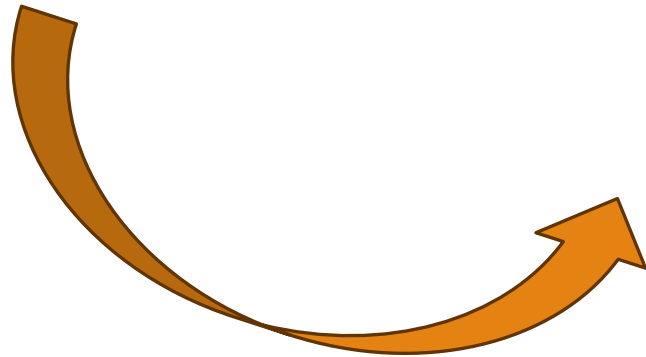
Attribute

Label

The table stores attributes. Pattern mining uses selected labels as items.

Shared-Bike Transaction Database

trip_id	member	period	start station	duration
T1	Subscriber	Morning	Central	Short
T2	Subscriber	Morning	Central	Short
T3	Subscriber	Morning	Riverside	Short
T4	Casual	Weekend	Riverside	Long
T5	Casual	Weekend	Museum	Long
T6	Casual	Weekend	Riverside	Long
T7	Subscriber	Evening	Central	Short
T8	Casual	Weekend	Museum	Long



TID	Items present
T1	{Subscriber, Morning, Central, Short}
T2	{Subscriber, Morning, Central, Short}
T3	{Subscriber, Morning, Riverside, Short}
T4	{Casual, Weekend, Riverside, Long}
T5	{Casual, Weekend, Museum, Long}
T6	{Casual, Weekend, Riverside, Long}
T7	{Subscriber, Evening, Central, Short}
T8	{Casual, Weekend, Museum, Long}

Transaction Containment and Support

- A transaction contributes to the support of X if it contains all items in X . Extra items are allowed.

$T1 = \{\text{Subscriber, Morning, Central, Short}\}$

Examples supported by $T1$

- {Subscriber}
- {Morning, Central}
- {Subscriber, Morning, Short}

Examples not supported by $T1$

- {Casual}
- {Central, Long}
- {Subscriber, Riverside}

A transaction with k items supports $2^k - 1$ non-empty subsets.

Basic Concepts: Frequent Itemsets (**Patterns**)

- An itemset (or a pattern) X is *frequent* if the support of X is **no less than** a *minsup threshold* σ
- Let $\sigma = 50\%$ (σ : *minsup* threshold), and for the given 5-transaction dataset
 - All the frequent 1-itemsets:
 - Beer: 3/5 (60%); Nuts: 3/5 (60%); Diaper: 4/5 (80%); Eggs: 3/5 (60%)
 - All the frequent 2-itemsets:
 - {Beer, Diaper}: 3/5 (60%)
 - All the frequent 3-itemsets?
 - None



Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Why do these itemsets (shown on the left) form the complete set of frequent k -itemsets (patterns) for any k ? (checking?)

- **Observation:** We need an efficient search method that prunes impossible candidates while preserving completeness.

The Minimum Support Threshold

TID	Items present
T1	{Subscriber, Morning, Central, Short}
T2	{Subscriber, Morning, Central, Short}
T3	{Subscriber, Morning, Riverside, Short}
T4	{Casual, Weekend, Riverside, Long}
T5	{Casual, Weekend, Museum, Long}
T6	{Casual, Weekend, Riverside, Long}
T7	{Subscriber, Evening, Central, Short}
T8	{Casual, Weekend, Museum, Long}

Lower minsup increases discovery sensitivity and computational cost.

minsup count	Patterns retained	Main risk
4	Very common pairs and triples	Miss useful but less common patterns
3	Core patterns in the bike example	Balanced teaching setting
2	Museum-related patterns also appear	More candidates and redundancy
1	Every observed combination	Pattern explosion

Quick Check: Support

TID	Items present
T1	{Subscriber, Morning, Central, Short}
T2	{Subscriber, Morning, Central, Short}
T3	{Subscriber, Morning, Riverside, Short}
T4	{Casual, Weekend, Riverside, Long}
T5	{Casual, Weekend, Museum, Long}
T6	{Casual, Weekend, Riverside, Long}
T7	{Subscriber, Evening, Central, Short}
T8	{Casual, Weekend, Museum, Long}

Question: For $X = \{\text{Casual, Weekend, Long}\}$, what is $s(X)$?

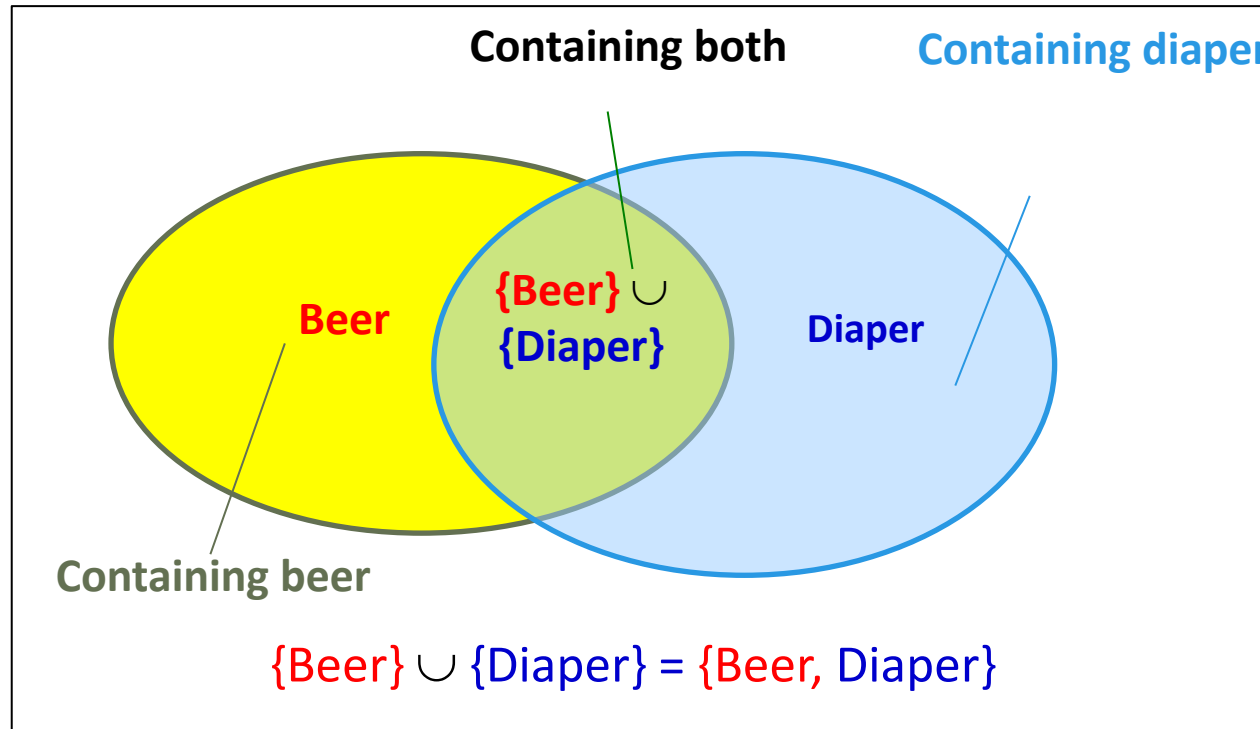
A	B	C	D
2/8	3/8	4/8	6/8

Answer: C.

Support counts transactions containing the complete itemset.

From Frequent Itemsets to **Association Rules**

- Compared with itemsets, *association rules* can be more telling
 - Association rules add *direction* and *conditional meaning* to frequent itemsets.
 - Ex. $Diaper \rightarrow Beer$, transactions containing Diaper are likely also to contain Beer.



$$\underbrace{\{Beer\} \cup \{Diaper\}}_{\text{union of items}} = \{Beer, Diaper\}$$
$$\underbrace{T_{Beer} \cap T_{Diaper}}_{\text{intersection of transactions}} = \text{transactions containing both}$$

Note: $X \cup Y$: the union of two itemsets. The set contains both X and Y.

Association Rules

- How do we *compute the strength* of an association rule $X \rightarrow Y$ (Both X and Y are itemsets, eg $Diaper \rightarrow Beer$)?

- We first compute the following *two metrics*, s and c .

- Support s of $X \cup Y$**

- Ex. $s\{Diaper, Beer\} = 3/5 = 0.6$ (i.e., 60%)

- Confidence c of $X \rightarrow Y$**

- The *conditional probability* that a transaction containing X also contains Y :

$$c = \text{sup}(X, Y) / \text{sup}(X)$$

- Ex. $c = \text{sup}\{Diaper, Beer\} / \text{sup}\{Diaper\} = 3/4 = 0.75$

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Support measures prevalence. Confidence measures conditional reliability.
Neither metric alone establishes association or causality.

Rule Direction Changes the Denominator

$$\text{conf}(X \Rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$$

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Rule	Joint count	Left-hand-side count	Confidence
Diaper $\Rightarrow$ Beer	3	4	$3/4 = 75\%$
Beer $\Rightarrow$ Diaper	3	3	$3/3 = 100\%$

Support is symmetric. Confidence is directional.

Mining Frequent Itemsets and Association Rules

□ Association rule mining

- Given two thresholds: *minsup*, *minconf*
- Find **all** of the rules, $X \rightarrow Y (s, c)$ such that $s \geq \text{minsup}$ and $c \geq \text{minconf}$

□ Let *minsup* = 50%

- Freq. 1-itemsets: Beer: 3, Nuts: 3, Diaper: 4, Eggs: 3
- Freq. 2-itemsets: {Beer, Diaper}: 3

□ Let *minconf* = 50%

- $\text{Beer} \rightarrow \text{Diaper}$ ($s=60\%$, $c=100\%$)
- $\text{Diaper} \rightarrow \text{Beer}$ ($s=60\%$, $c=75\%$)



Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

□ Observations:

- Mining association rules and mining frequent patterns are very close problems
- **Scalable methods** are needed for mining large datasets

(Q: Are these all the rules satisfying the two conditions?)

Association-Rule Mining as Two Stages



- ① Support pruning operates on itemsets.
- ② Confidence filtering operates on directional rules.

Separate pattern discovery from rule evaluation.

Support, Confidence, and Interpretation

Measure	Formula	Question
Rule support	$s(X \cup Y)$	How much of the whole database does the rule cover ?
Confidence	$s(X \cup Y) / s(X)$	Among transactions with X, how often does Y occur?
Lift (later)	$\text{conf}(X \Rightarrow Y) / s(Y)$	How does confidence compare with Y's base rate?

Note: Association does not imply causation.

Challenge: There Are **Too Many** Frequent Patterns!

- ❑ A long pattern contains a combinatorial number of sub-patterns
- ❑ How many frequent itemsets does the following TDB₁ contain (minsup = 1)?
 - ❑ TDB₁: T₁: {a₁, ..., a₅₀}; T₂: {a₁, ..., a₁₀₀}
 - ❑ Let's have a try
 - 1-itemsets: {a₁}: 2, {a₂}: 2, ..., {a₅₀}: 2, {a₅₁}: 1, ..., {a₁₀₀}: 1,
 - 2-itemsets: {a₁, a₂}: 2, ..., {a₁, a₅₀}: 2, {a₁, a₅₁}: 1 ..., ..., {a₉₉, a₁₀₀}: 1,
 - ..., ..., ..., ...
 - 99-itemsets: {a₁, a₂, ..., a₉₉}: 1, ..., {a₂, a₃, ..., a₁₀₀}: 1
 - 100-itemset: {a₁, a₂, ..., a₁₀₀}: 1
- ❑ The total number of frequent itemsets:

$$\binom{100}{1} + \binom{100}{2} + \binom{100}{3} + \cdots + \binom{100}{100} = 2^{100} - 1$$

A too huge set for any one to compute or store!

Combinatorial Search Space

- The database may be small while the pattern space remains enormous.

Number of items d	Possible non-empty itemsets $2^d - 1$
10	1,023
20	1,048,575
50	$> 10^{15}$
100	$> 10^{30}$

Efficient mining depends on *compact representations* and *pruning*.

Expressing Patterns in Compressed Form: **Closed Patterns**

How to handle such a challenge?

- **Solution 1: Closed patterns:** A pattern (itemset) X is **closed** if X is *frequent*, and there exists *no super-pattern* $Y \supset X$, with the same support as X
 - Let Transaction TDB₁: $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - Suppose $minsup = 1$. How many closed patterns does TDB₁ contain?
 - Two: $P_1: \{\{a_1, \dots, a_{50}\}: 2\}$; $P_2: \{\{a_1, \dots, a_{100}\}: 1\}$
- **Closed pattern** is a **lossless compression** of frequent patterns
 - Reduces the # of patterns but **does not lose the support information!**
 - You will still be able to say: $\{\{a_2, \dots, a_{40}\}: 2\}$, $\{\{a_5, a_{51}\}: 1\}$

Expressing Patterns in Compressed Form: **Max-Patterns**

- **Solution 2: Max-patterns:** A pattern X is a **max-pattern** if X is frequent and there exists no frequent super-pattern $Y \supset X$
- Different from closed patterns?
 - Do not care the real support of the sub-patterns of a max-pattern
 - Let Transaction DB TDB_1 : $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - Suppose $minsup = 1$. How many max-patterns does TDB_1 contain?
 - One: $P: \{\{a_1, \dots, a_{100}\}: 1\}$
- **Max-pattern** is a **lossy compression**!
 - We only know $\{a_1, \dots, a_{40}\}$ is frequent
 - But we do not know the real support of $\{a_1, \dots, a_{40}\}$, ..., any more!
- In many applications, mining close-patterns is more desirable than mining max-patterns

Closed and Maximal Frequent Patterns

Property	Closed frequent pattern	Maximal frequent pattern
Definition	No proper superset has the same support	No proper superset is frequent
Information retained	Exact supports are recoverable	Only frequency status is recoverable
Compression	Usually more patterns	Usually fewer patterns
Best use when	Exact support values matter	Only the frequent frontier matters

Every maximal frequent pattern is a closed frequent pattern, but not every closed frequent pattern is maximal.

Maximal frequent patterns $\subseteq$ Closed frequent patterns $\subseteq$ Frequent patterns

Menti Check: Closed or Maximal?

Question: X has support 5. No proper superset of X has support 5, but at least one proper superset has support 3 and is still frequent. What is true?

A	B	C	D
X is maximal only	X is closed but not maximal	X is neither	X must have support 3

Submit your answer through Menti!

Answer: B

Equal support matters for closedness. Any frequent superset matters for maximality.

Efficient Pattern Mining Methods

- The Downward Closure Property of Frequent Patterns
 - The Apriori Algorithm 
 - Extensions or Improvements of Apriori
- Mining Frequent Patterns by Exploring Vertical Data Format
- FPGrowth: A Frequent Pattern-Growth Approach
- Mining Closed Patterns

The Downward Closure Property of Frequent Patterns

- Observation: From $TDB_1: T_1: \{a_1, \dots, a_{50}\}; T_2: \{a_1, \dots, a_{100}\}$
 - We get a frequent itemset: $\{a_1, \dots, a_{50}\}$
 - Also, its subsets are all frequent: $\{a_1\}, \{a_2\}, \dots, \{a_{50}\}, \{a_1, a_2\}, \dots, \{a_1, \dots, a_{49}\}, \dots$
 - There are some hidden relationships among frequent patterns!
- The **downward closure** (also called “Apriori”) property of frequent patterns
 - Apriori: Any subset of a frequent itemset must be frequent
 - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
 - Every transaction containing {beer, diaper, nuts} also contains {beer, diaper}
- **Efficient mining methodology**
 - If any subset of an itemset S is infrequent, then there is no chance for S to be frequent — why do we even have to consider S !?

A sharp knife for pruning!

Support Monotonicity

If $Y \subset X$, then $\text{support}(Y) \geq \text{support}(X)$

Itemset	Matching bike trips	Support count
{Subscriber}	T1, T2, T3, T7	4
{Subscriber, Morning}	T1, T2, T3	3
{Subscriber, Morning, Central}	T1, T2	2

Confidence does not have this anti-monotone property.

The Apriori Pruning Rule



$\{\text{Morning, Central}\} \subset \{\text{Subscriber, Morning, Central}\}$

One infrequent subset is enough to reject the whole candidate.

Apriori Pruning and Scalable Mining Methods

- ❑ Apriori pruning principle: If there is any itemset which is infrequent, its superset should not even be generated! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)

- ❑ **Scalable mining Methods: Three major approaches**
 - ❑ Level-wise, join-based approach: **Apriori** (Agrawal & Srikant@VLDB'94)
 - ❑ Vertical data format approach: **Eclat** (Zaki, Parthasarathy, Ogiwara, Li @KDD'97)
 - ❑ Frequent pattern projection and growth: **FPgrowth** (Han, Pei, Yin @SIGMOD'00)

Apriori: A Candidate Generation & Test Approach

- Outline of **Apriori** (level-wise, candidate generation and test)
 - **Scan** DB once to get frequent 1-itemset
 - **Repeat**
 - Generate length-($k+1$) candidate itemsets from length- k frequent itemsets
 - Test the candidates against DB to find frequent ($k+1$)-itemsets at *minsup*
 - Set $k := k + 1$
 - **Until** no frequent or candidate set can be generated
 - Return all the frequent itemsets derived

The Apriori Algorithm (Pseudo-Code)

C_k : Candidate itemset of size k

F_k : Frequent itemset of size k

$K := 1$;

$F_k := \{\text{frequent items}\}$; // frequent 1-itemset

While ($F_k \neq \emptyset$) **do** { // when F_k is non-empty

$C_{k+1} := \text{candidates generated from } F_k$; // candidate generation

 Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;

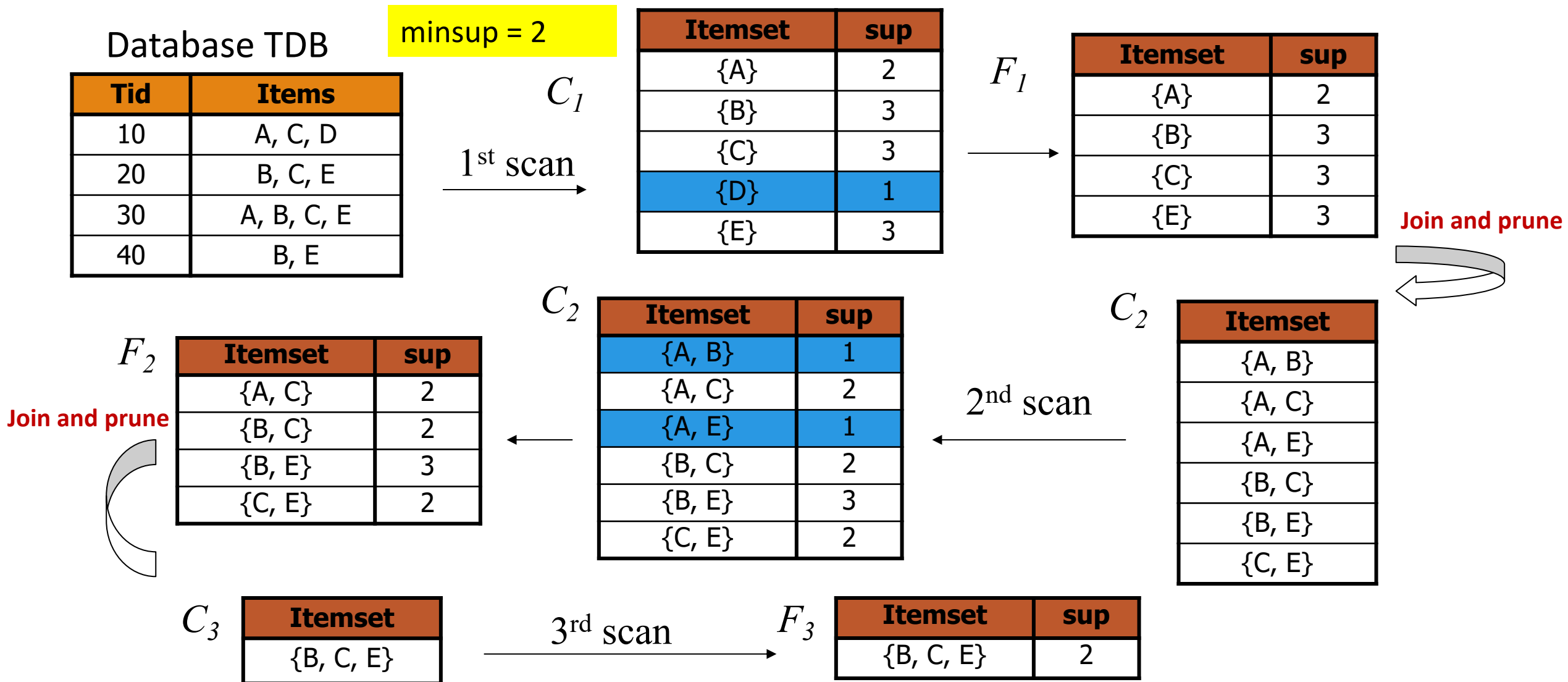
$k := k + 1$

}

return $\cup_k F_k$ // return F_k generated at each level

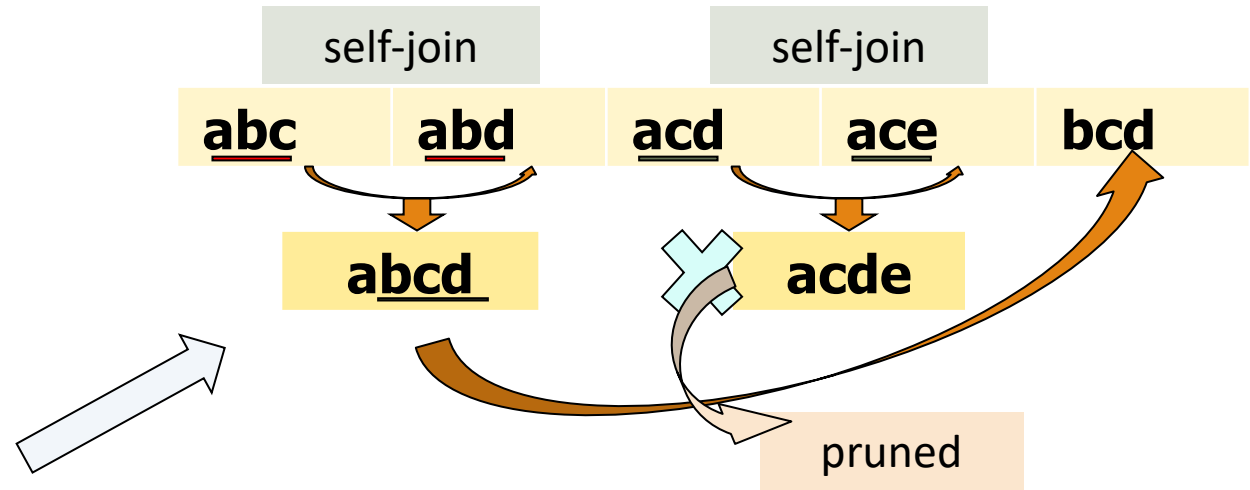
Apriori searches *breadth* first, one itemset length at a time.

The Apriori Algorithm—An Example



Apriori: Implementation Tricks

- How to generate candidates?
 - Step 1: self-joining F_k
 - Step 2: pruning
- Example of candidate-generation
 - $F_3 = \{abc, abd, acd, ace, bcd\}$
 - Self-joining: $F_3 * F_3$**
 - $abcd$ from abc and abd
 - $acde$ from acd and ace
 - Pruning:**
 - $acde$ is removed because ade is not in F_3
 - $C_4 = \{abcd\}$



Self-join does not mean joining every pair in F_k . We join two itemsets only when their first $(k-1)$ ordered items are identical. This generates each candidate once and avoids duplicates.

Apriori: Strengths and Limits

Strengths

- ✓ Simple level-wise search
- ✓ Complete under a fixed minsup
- ✓ Transparent pruning logic
- ✓ Works well for many sparse datasets

VS

Limits

- Large candidate sets
- Repeated database scans
- High memory for intermediate counts
- Slow on dense data or low minsup

Part 1 Summary and Part 2 Question

What we can now do

- ✓ Define transactions and itemsets
- ✓ Compute support and confidence
- ✓ Trace Apriori join and prune
- ✓ Explain closed and maximal patterns

What remains

- Avoid repeated scans
- Avoid large candidate tables
- Compress repeated transaction prefixes
- Evaluate rules beyond confidence

Question for part 2:

Can a different representation make mining faster and the output more useful?

Appendix

Worked Examples for Part 1

Apriori Walkthrough: Data and Threshold

TID	Items
T1	Subscriber, Morning, Central, Short
T2	Subscriber, Morning, Central, Short
T3	Subscriber, Morning, Riverside, Short
T4	Casual, Weekend, Riverside, Long
T5	Casual, Weekend, Museum, Long
T6	Casual, Weekend, Riverside, Long
T7	Subscriber, Evening, Central, Short
T8	Casual, Weekend, Museum, Long

Use minsup count = 3.

Pass 1: C₁ and L₁

Item	Count	Keep?	Item	Count	Keep?
Subscriber	4	Yes	Casual	4	Yes
Short	4	Yes	Long	4	Yes
Morning	3	Yes	Weekend	4	Yes
Central	3	Yes	Riverside	3	Yes
Museum	2	No	Evening	1	No

Only frequent singletons can participate in larger frequent itemsets.

Pass 2: Candidate Pairs C₂

Representative candidate	Count	Retain?
{Subscriber, Short}	4	Yes
{Subscriber, Morning}	3	Yes
{Morning, Central}	2	No
{Casual, Weekend}	4	Yes
{Casual, Riverside}	2	No
{Weekend, Long}	4	Yes

Counting converts C₂ into the frequent set L₂.

Frequent Pairs L₂

Subscriber-side pairs

- {Subscriber, Short}: 4
- {Subscriber, Morning}: 3
- {Subscriber, Central}: 3
- {Morning, Short}: 3
- {Central, Short}: 3

VS

Casual-side pairs

- {Casual, Weekend}: 4
- {Casual, Long}: 4
- {Weekend, Long}: 4
- Riverside has no frequent pair

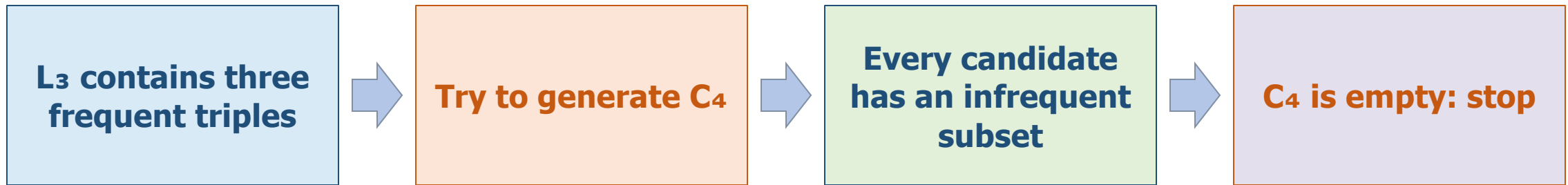
Frequent singletons do not automatically create frequent pairs.

Join and Prune Candidate Triples

Candidate	Required 2-subsets	Decision
{Subscriber, Morning, Short}	All three pairs are in L_2	Generate and count
{Subscriber, Morning, Central}	{Morning, Central} is absent from L_2	Prune
{Casual, Weekend, Long}	All three pairs are in L_2	Generate and count

The subset test removes impossible triples before support counting.

Stopping Condition and Completeness



Frequent output = $L_1 \cup L_2 \cup L_3$

Downward closure guarantees that Apriori does not prune a truly frequent itemset.

Rules from a Frequent Itemset

F = {Casual, Weekend, Long}, support = 4/8

Left-hand side	Right-hand side	Confidence
Casual	Weekend, Long	$4/4 = 1.00$
Weekend	Casual, Long	$4/4 = 1.00$
Long	Casual, Weekend	$4/4 = 1.00$
Casual, Weekend	Long	$4/4 = 1.00$

minsup controls itemset discovery. **minconf** controls rule output.

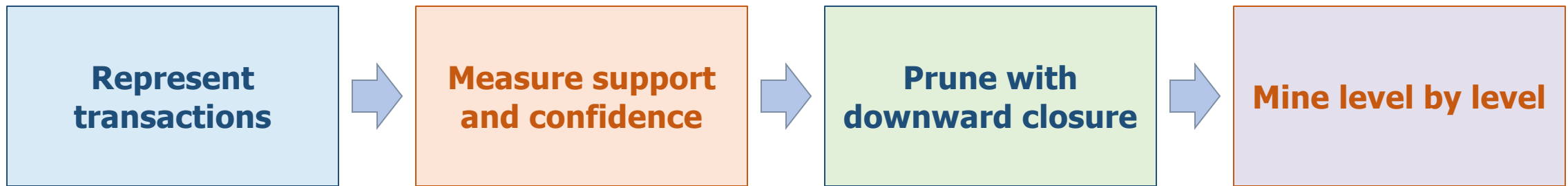
Quick Check: Apriori

Question: Why can Apriori prune $\{A, B, C\}$ when $\{A, B\}$ is infrequent?

A	B	C	D
Confidence decreases	Support cannot increase when C is added	Lift becomes zero	The database is sparse

Answer: B. A superset cannot have higher support than its subset.

Part 1 Knowledge Map



The central idea is to avoid counting candidates that cannot be frequent.

Recommended Readings (Basic Concepts)

- ❑ R. Agrawal, T. Imielinski, and A. Swami, “Mining association rules between sets of items in large databases”, in Proc. of SIGMOD'93
- ❑ R. J. Bayardo, “Efficiently mining long patterns from databases”, in Proc. of SIGMOD'98
- ❑ N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal, “Discovering frequent closed itemsets for association rules”, in Proc. of ICDT'99
- ❑ J. Han, H. Cheng, D. Xin, and X. Yan, “Frequent Pattern Mining: Current Status and Future Directions”, Data Mining and Knowledge Discovery, 15(1): 55-86, 2007

Part 2: Mine, Compress, and Evaluate

Mine	Compress	Evaluate
ECLAT and FP-Growth	Closed and maximal itemsets	Base rates, lift, χ^2 , and null-invariant measures

The goal is faster mining, smaller output, and better interpretation.

Retrieval Check: Downward Closure

Which statement justifies pruning $\{A, B, C, D\}$ before counting it?

A	B	C	D
$\{A,B\}$ is frequent	$\{A,B,C\}$ is infrequent	A has high confidence	D has high lift

Answer: **B.**

Any infrequent subset makes every superset infrequent.

The Apriori Algorithm (Pseudo-Code *again*)

C_k : Candidate itemset of size k

F_k : Frequent itemset of size k

$K := 1$;

$F_k := \{\text{frequent items}\}$; // frequent 1-itemset

While ($F_k \neq \emptyset$) **do** { // when F_k is non-empty

$C_{k+1} := \text{candidates generated from } F_k$; // candidate generation

 Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;

$k := k + 1$

}

return $\cup_k F_k$ // return F_k generated at each level

Apriori searches *breadth* first, one itemset length at a time.

Apriori: Improvements and Alternatives

❑ Reduce passes of transaction database scans

- ❑ **Partitioning** (e.g., Savasere, et al., 1995)
- ❑ Dynamic itemset counting (Brin, et al., 1997)

To be discussed in subsequent slides

❑ Shrink the number of candidates

- ❑ **Hashing** (e.g., DHP: Park, et al., 1995)
- ❑ Pruning by support lower bounding (e.g., Bayardo 1998)
- ❑ Sampling (e.g., Toivonen, 1996)

To be discussed in subsequent slides

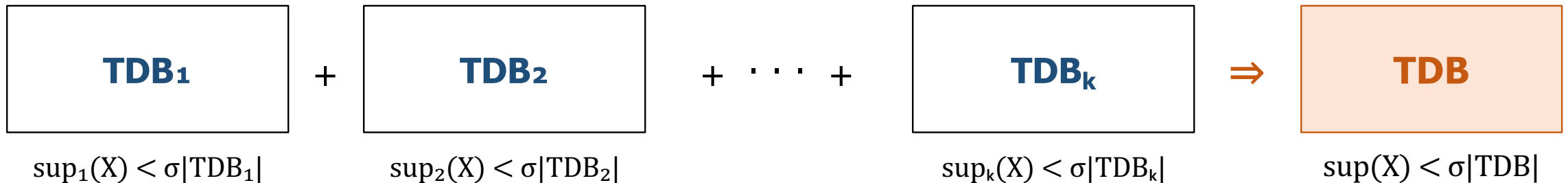
❑ Exploring special data structures

- ❑ Tree projection (Agarwal, et al., 2001)
- ❑ H-miner (Pei, et al., 2001)
- ❑ Hypercube decomposition (e.g., LCM: Uno, et al., 2004)

Partitioning: Scan the Database **Only Twice**

- **Theorem:** *Any globally frequent itemset in TDB must be frequent in at least one partition.*

σ denotes the minimum relative support threshold.



Why the theorem holds

Assume X is infrequent in every partition: **Sum the local support counts:**

$$\sup_i(X) < \sigma|TDB_i| \quad \text{for every } i$$

$$\sup_D(X) = \sum_i \sup_i(X) < \sigma \sum_i |TDB_i| = \sigma|TDB|$$

Therefore X is globally infrequent. The contrapositive proves the theorem.

The union of local frequent sets contains every globally frequent itemset.

Partitioning: Scan the Database **Only Twice**

SCAN 1 Local mining

- 1 Partition the database**
Choose chunks that **fit in main memory**.
- 2 Mine each partition**
Find the local frequent set L_i using threshold $\sigma|TDB_i|$.
- 3 Form the candidate union**
 $C = L_1 \cup L_2 \cup \dots \cup L_k$

Why fit in memory? Local mining may use several RAM passes without another full disk scan.

SCAN 2 Global verification

- 1 Scan the complete TDB**
Process every transaction one more time.
- 2 Count exact global support**
Count $\text{supd}(X)$ only for candidates $X \in C$.
- 3 Apply the global threshold**
 $F = \{X \in C : \text{supd}(X) \geq \sigma|TDB|\}$

C may include false positives. The theorem guarantees that C has no false negatives.

**The method performs two full disk scans.
Local mining may use multiple in-memory passes.**

Direct Hashing and Pruning (DHP)

- ❑ **First scan:** count 1-itemsets and hash every 2-itemset for each transaction.
- ❑ **For each transaction T:**
 1. Enumerate every 2-itemset X in T
 2. $b = \text{hash}(X)$
 3. $\text{bucketCount}[b] = \text{bucketCount}[b] + 1$

Example:

$T1 = \{a, b, d\}$ contains $\{ab, ad, bd\}$

$h(ab) = h(ad) = V1$, while $h(bd) = V2$

T1 adds 2 to V1 and 1 to V2.

$T2 = \{c, e\}$, $h(ce) = V1$, so T2 adds 1 to V1.

After the full first scan

Bucket	Examples of pairs mapped here	Count
V1	{ab, ad, ce, ...}	35
V2	{bd, be, de, ...}	298
...	...	...
Vn	{yz, qs, wt, ...}	58

Bucket count = total occurrences of all 2-itemsets mapped to that bucket.

Note: Different itemsets may hash to the same bucket.

Direct Hashing and Pruning (DHP)

- ❑ After the first scan, compare each bucket count with *minsup* (σ).

$$\text{support}(X) \leq \text{bucketCount}[\text{hash}(X)]$$

The bucket includes X and *every colliding itemset*, so its count is an **upper bound** on $\text{support}(X)$.

$$\text{minsup}(\sigma) = 80$$

σ denotes the minimum support count.

- ❑ Remaining pairs still require exact support counting in the second scan.
- ❑ Extend to k -itemsets.

Bucket	Mapped itemsets	Count	Decision
V1	{ab, ad, ce, ...}	35	Prune all
V2	{bd, be, de, ...}	298	Keep as candidates
Vn	{yz, qs, wt, ...}	58	Prune all

A low bucket count ($< \text{minsup}$) guarantees infrequency.

A high bucket count ($\geq \text{minsup}$) indicates possibility only.

Apriori Improvements and Alternatives

Goal	Methods	Main idea
Reduce database scans	Partitioning, dynamic itemset counting	Count more candidates per pass or verify only a reduced union
Reduce candidates	DHP, support lower bounds, sampling	Reject or approximate candidates before exact counting
Change the search structure	Tree projection, H-Miner, LCM	Use compact structures and specialized traversal

All methods preserve or approximate the same frequent-itemset objective; they reduce different costs.

Exploring Vertical Data Format: ECLAT

- ❑ ECLAT (Equivalence Class Transformation): A **depth-first search** algorithm using set intersection [Zaki et al. @KDD'97]
- ❑ **Vertical format:** $t(X) = \{\text{transaction IDs containing } X\}$
 - ❑ Properties of **Tid-Lists**
 - ❑ $t(X) = t(Y)$: X and Y always happen together (e.g., $t(ac) = t(d)$)
 - ❑ $t(X) \subseteq t(Y)$: transaction having X always has Y (e.g., $t(ac) \subseteq t(ce)$)
- ❑ **Frequent patterns: vertical intersections**
 - ❑ Using **diffset** to accelerate mining
 - ❑ Store the TIDs *removed from* the **parent Tid-list**.
 - ❑ $t(e) = \{T10, T20, T30\}$, $t(ce) = \{T10, T30\} \rightarrow \text{Diffset}(ce, e) = \{T20\}$

A transaction DB in Horizontal Data Format

Tid	Itemset
10	a, c, d, e
20	a, b, e
30	b, c, e

The transaction DB in Vertical Data Format

Item	TidList
a	10, 20
b	20, 30
c	10, 30
d	10
e	10, 20, 30

ECLAT Support by TID-Set Intersection

Item	TID set
Subscriber	{T1, T2, T3, T7}
Morning	{T1, T2, T3}
Subscriber $\cup$ Morning	{T1, T2, T3}

$$t(\text{Subscriber} \cup \text{Morning}) = t(\text{Subscriber}) \cap t(\text{Morning})$$

The intersection contains three TIDs, so the support count is 3.

ECLAT Search Space: Equivalence Classes

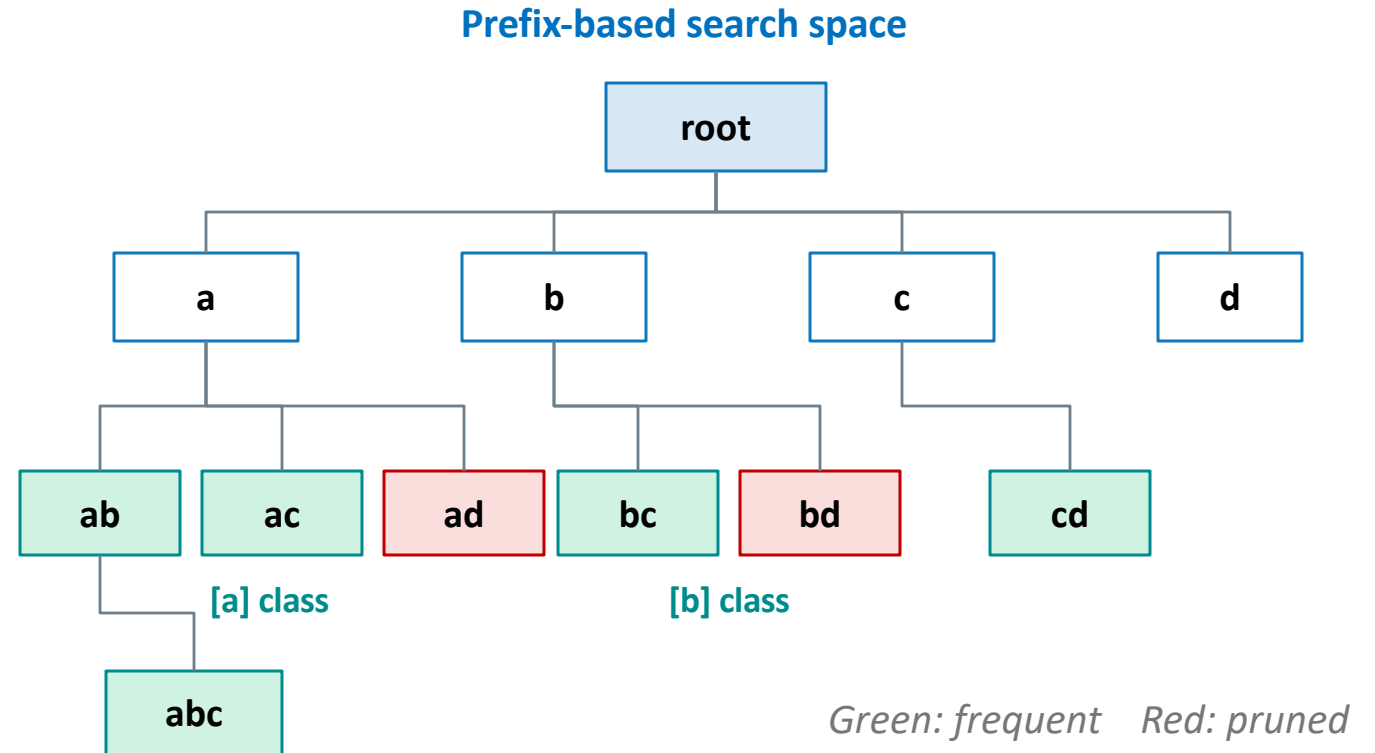
- Fix an item order: $a < b < c < d$. Extend each prefix using only later items.

Example database (minsup = 2)

T1: {a,b,c} T2: {a,c,d}

T3: {b,c,d} T4: {a,b,c}

Item	TID-list	Support
a	{T1, T2, T4}	3
b	{T1, T3, T4}	3
c	{T1, T2, T3, T4}	4
d	{T2, T3}	2



Itemsets with the same prefix form an equivalence class; **ECLAT explores one class depth first.**

ECLAT: Depth-First Pattern Extension

- **Core operation:** extend a prefix and intersect the TID-lists of its extensions.

Recursive procedure

ECLAT(P, E)

1. for each extension X_i in E :
2. $Q = P \text{ union } \{X_i\}$
3. output Q
4. for each X_j after X_i :
5. $T(Q \text{ union } X_j) = T(Q)$
 intersect $T(X_j)$
6. if support $\geq$ minsup:
 keep the extension
7. recursively mine the extensions of Q

Worked example: prefix a

Pattern	TID-list	Sup.	Decision
ab	{T1, T4}	2	Frequent
ac	{T1, T2, T4}	3	Frequent
ad	{T2}	1	Prune
abc	$t(ab) \text{ intersect } t(ac) = \{T1, T4\}$	2	Frequent

Because ad is infrequent, extensions abd and acd are pruned.

Frequent itemsets: {a}, {b}, {c}, {d}, {ab}, {ac}, {bc}, {cd}, and {abc}

ECLAT Diffsets and Practical Trade-offs

- ❑ ECLAT replaces repeated database scans with TID-list intersections.
- ❑ Diffsets store only what changes between a parent (prefix) and its extension.

Diffset: $\Delta(e \rightarrow ce) = t(e) - t(ce)$

Set	TIDs
$t(e)$	{T10, T20, T30}
$t(ce)$	{T10, T30}
Δ	{T20}

Helpful when

- TID sets fit in memory
- Bitsets or intersections are fast
- Depth-first search limits active state

Main risk

Long Tid lists and a genuinely large pattern space

Apriori and ECLAT

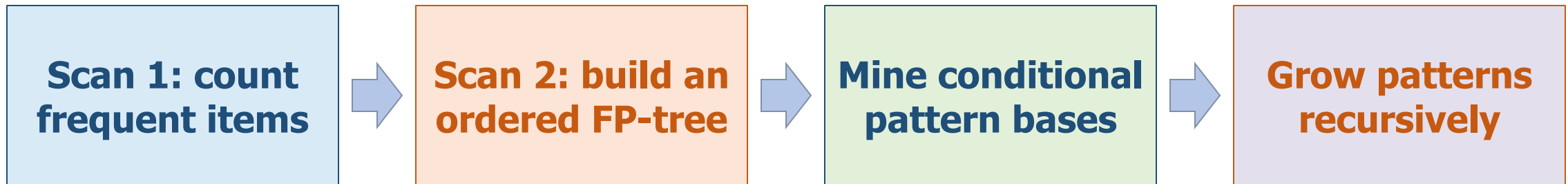
Dimension	Apriori	ECLAT
Data format	Horizontal transactions	Vertical TID sets
Search	Breadth first	Usually depth first
Core operation	Generate and count	Intersect TID sets
Typical bottleneck	Candidates and repeated scans	Intersection memory and long TID lists

**Representation changes the bottleneck;
it does not remove combinatorial growth.**

Why Mining Frequent Patterns by Pattern Growth?

- Apriori: A *breadth-first search* mining algorithm
 - First find the complete set of frequent k -itemsets
 - Then derive frequent $(k+1)$ -itemset candidates
 - Scan DB again to find true frequent $(k+1)$ -itemsets
- Motivation: Can we develop a *depth-first search* mining algorithm?
 - For a frequent itemset p , can subsequent search be confined to only *those transactions* that containing p ?
- Such thinking leads to a frequent pattern growth approach: **FPGrowth**

FP-Growth Overview



FP-Growth avoids global candidate levels C_k .

Example: From Transactional DB to Ordered Frequent Itemlist

Example: A Sample Transactional Database

TID	Items in the Transaction
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

Let min_support = 3

- Scan DB once, find single item frequent pattern: f:4, a:3, c:4, b:3, m:3, p:3
- Sort frequent items in *frequency descending order: f-list* F-list = f-c-a-b-m-p
- Scan DB again, use the ordered frequent itemlist for each transaction to construct *an FP-tree*

TID	Items in the Transaction	Ordered, frequent itemlist
100	{f, a, c, d, g, i, m, p}	f, c, a, m, p
200	{a, b, c, f, l, m, o}	f, c, a, b, m
300	{b, f, h, j, o, w}	f, b
400	{b, c, k, s, p}	c, b, p
500	{a, f, c, e, l, p, m, n}	f, c, a, m, p

Why FP-Growth Uses **One Global Order**

single item frequent pattern

f:4, a:3, c:4, b:3, m:3, p:3

During the second database scan, for each transaction, **remove infrequent items**

Order frequent items

F-list = f-c-a-b-m-p



TID	Original items in Transaction
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

Consistent ordering turns common item combinations into shared prefixes.

Example: Construct **FP-tree** from Transaction DB

TID	Ordered, frequent itemlist
100	<u>f, c, a, m, p</u>
200	<u>f, c, a, b, m</u>
300	f, b
400	c, b, p
500	f, c, a, m, p

After inserting the 1st frequent Itemlist:
"f, c, a, m, p"

Item	Frqncy	hdr
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

Header Table

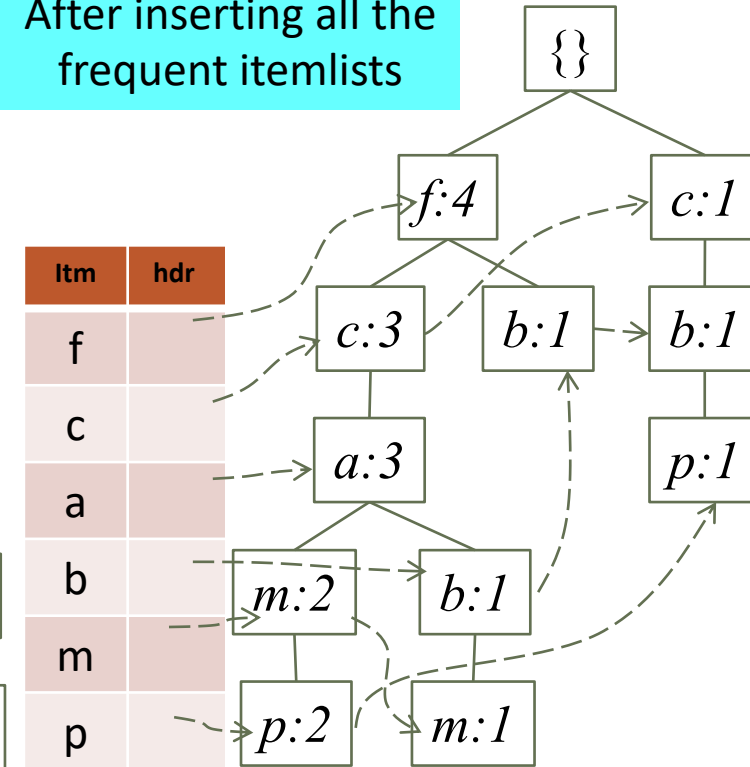
After inserting the 2nd frequent itemlist
"f, c, a, b, m"

Itm	hdr
f	
c	
a	
b	
m	
p	

F-list = f-c-a-b-m-p

f:4, a:3, c:4, b:3, m:3, p:3

After inserting all the frequent itemlists

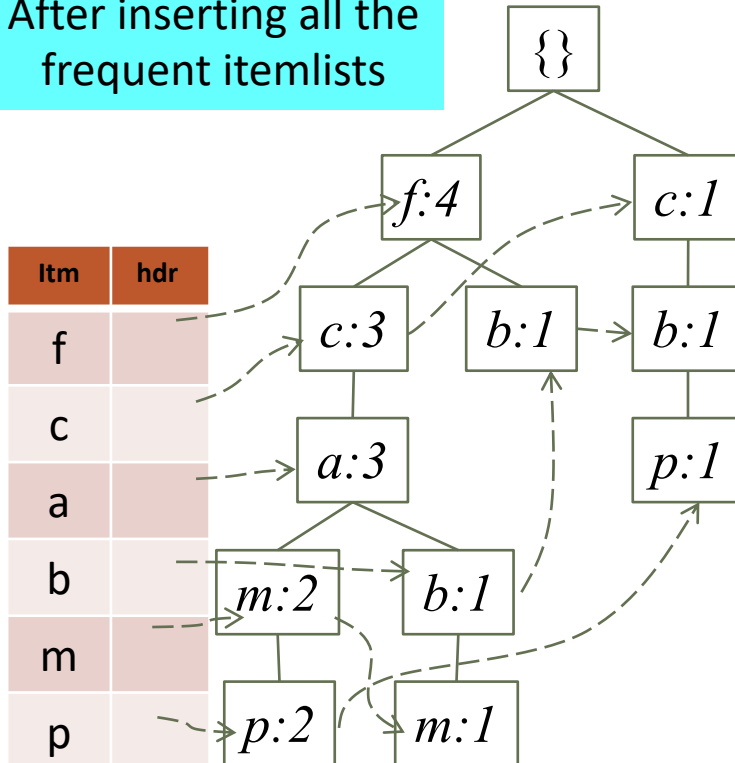


FP-Tree Construction:

- For each transaction, insert its ordered frequent-item list into the FP-tree.
- Reuse the common prefix, increment its node counts, and create new nodes for the remaining suffix.

Reading the Completed FP-Tree

After inserting all the frequent itemlists



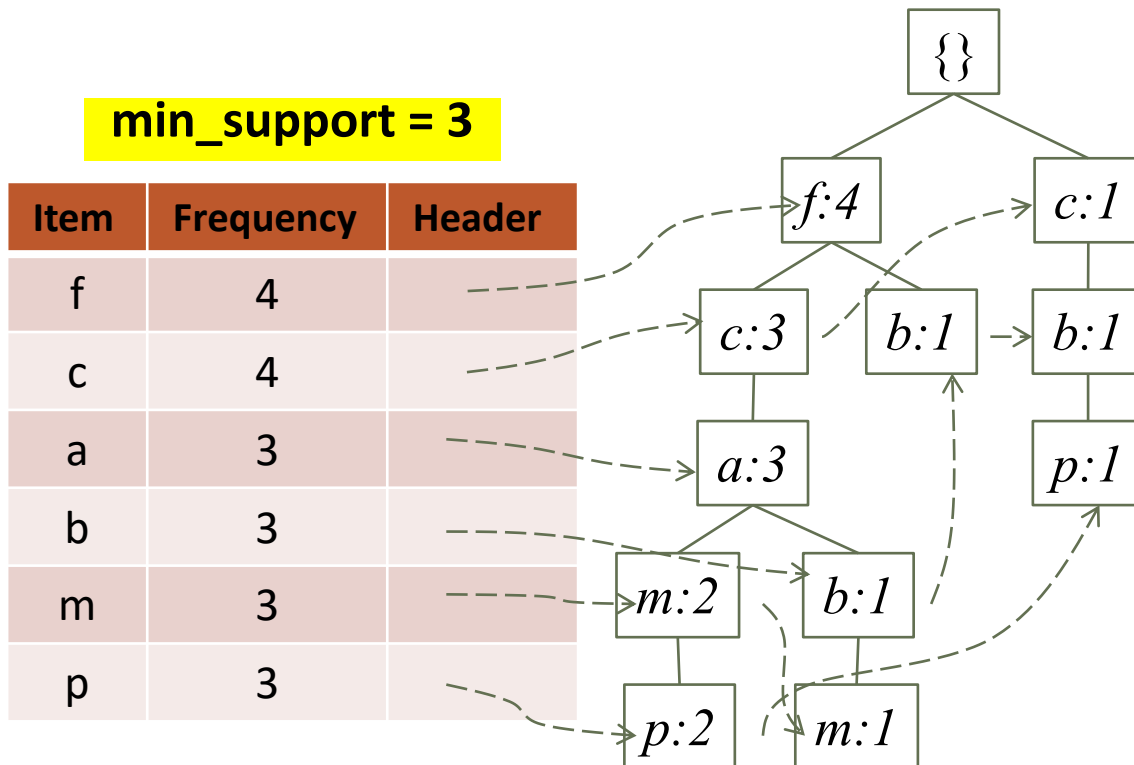
Tree element	Meaning
Path $f:4 \rightarrow c:3 \rightarrow a:3$	Three transactions share prefix f-c-a; four start with f
Two different b nodes	b follows different prefixes, so the nodes cannot merge
Node count	Number of transactions that reach the complete prefix
Header link	Connects all nodes for the same item during conditional mining

The FP-tree is a lossless index for frequent-pattern mining after infrequent items are removed.

Mining FP-Tree:

Divide and Conquer Based on Patterns and Data

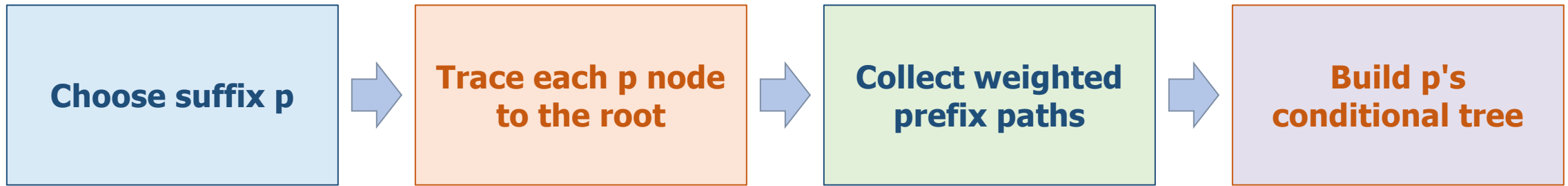
- Pattern mining can be partitioned according to current patterns
 - **p 's conditional database** (i.e., the database under the condition that p exists):
transformed prefix paths of item p (fixing p as *suffix*)
 - For suffix p , collect all prefix paths ending at p :
 p 's conditional pattern base = $\{fcam:2, cb:1\}$



Conditional Pattern Base of Each Suffix Item

<i>Item</i>	<i>Conditional database</i>
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

Conditional Pattern Base



Prefix path before p	Path count
f, c, a, m	2
c, b	1

Conditional mining replaces one global problem with smaller projected problems.

Mine Each Conditional Database Recursively

min_support = 3

<i>item</i>	<i>cond. data base</i>
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

□ For each conditional database

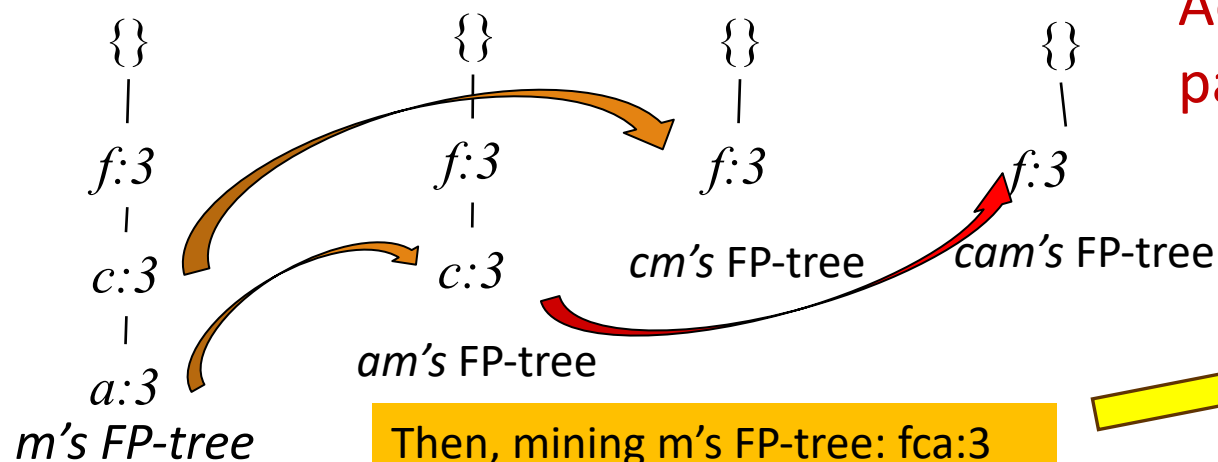
□ Mine single-item patterns

□ Construct its conditional FP-tree and recursively mine patterns with the current suffix

p's conditional DB: *fcam:2, cb:1* → *c:3*

m's conditional DB: *fca:2, fcab:1* → *fca:3*

b's conditional DB: *fca:1, f:1, c:1* → ϕ

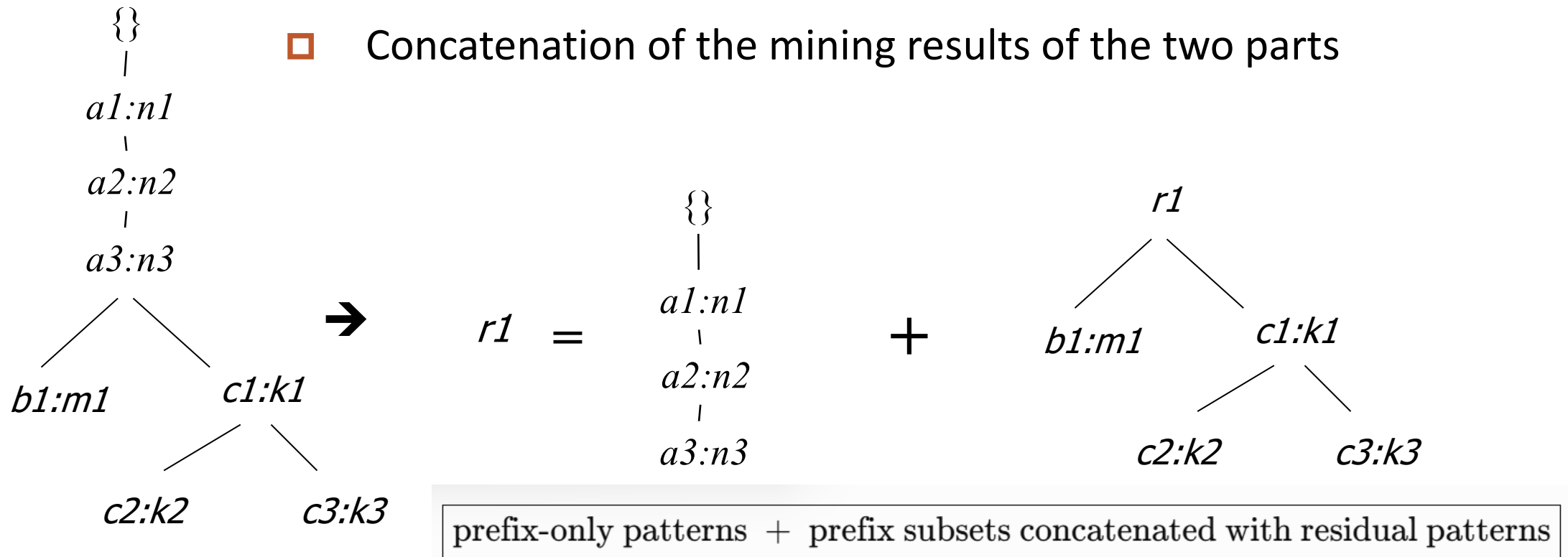


Actually, for single branch FP-tree, all the frequent patterns can be generated in one shot

m: 3
fm: 3, cm: 3, am: 3
fcm: 3, fam:3, cam: 3
fcam: 3

A Special Case: Single Prefix Path in FP-tree

- Suppose a (conditional) FP-tree T has a shared single prefix-path P
- Mining can be decomposed into two parts
 - Reduction of the single prefix path into one node
 - Concatenation of the mining results of the two parts



FPGrowth: Mining Frequent Patterns by Pattern Growth

□ Essence of frequent pattern growth (FPGrowth) methodology

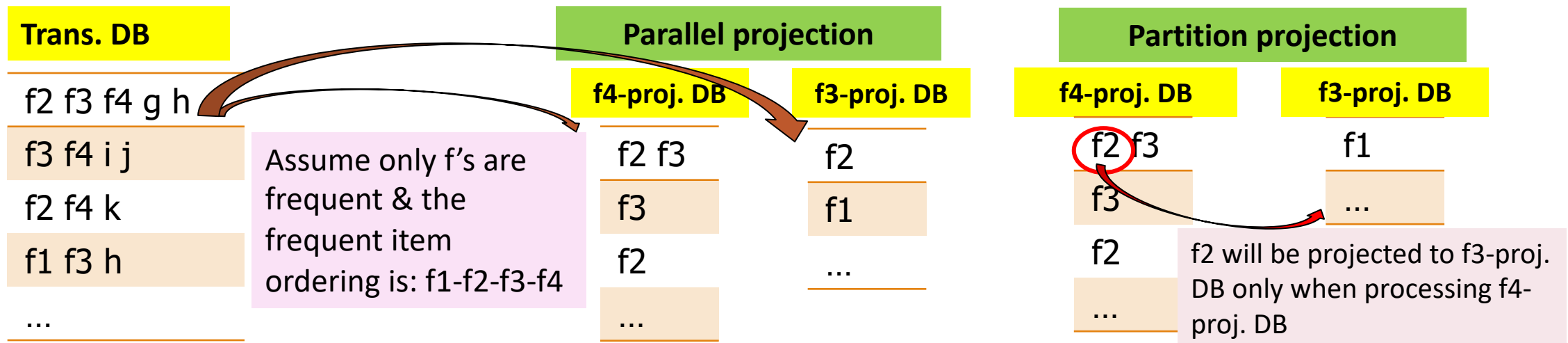
- Find frequent single items and partition the database based on each such single item pattern
- Recursively grow frequent patterns by doing the above for each *partitioned database* (also called the pattern's *conditional database*)
- To facilitate efficient processing, an efficient data structure, **FP-tree**, can be constructed

□ Mining becomes

- Recursively construct and mine (conditional) FP-trees
- Until the resulting FP-tree is empty, or until it contains only one path—*single path will generate all the combinations of its sub-paths, each of which is a frequent pattern*

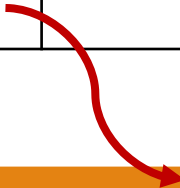
Scaling FP-growth by Item-Based Data Projection

- What if FP-tree cannot fit in memory? — *Do not construct the global FP-tree*
 - “Project” the database based on frequent single items
 - Construct & mine FP-tree for each projected DB
- Parallel projection vs. partition projection
 - Parallel projection: Project the DB on each frequent item
 - Space costly, all partitions can be processed in parallel
 - Partition projection: Partition the DB in order
 - Passing the unprocessed parts to subsequent partitions



Three Frequent-Itemset Mining Families

Dimension	Apriori	ECLAT	FP-Growth
Representation	Horizontal	Vertical TID sets	FP-tree
Search	Breadth first	Usually depth first	Pattern growth
Main operation	Generate and test	Set intersection	Conditional projection
Often effective when	Sparse data and moderate minsup	TID sets fit memory	Ordered prefixes compress well



Algorithm choice follows the representation and bottleneck.

Quick Check: FP-Tree Compression

Question : Why do repeated transactions compress in an FP-tree?

A	B	C	D
Items follow one global order	Confidence is anti-monotone	The tree stores only rules	Every node has a unique item

Answer: A.

Common prefixes align only under consistent ordering.

Frequent-Pattern Output Can Also Be Too Large

A compact FP-tree can still represent an exponential number of valid patterns.

Compact Mining Structure

- Shared prefixes compress transactions
- FP-growth avoids candidate generation
- The tree may fit comfortably in memory

Large Valid Output

- Every frequent pattern may require output
- Long transactions create many subsets
- Low min_sup further enlarges the result

If d items always occur together: # frequent itemsets = $2^d - 1$

Closed and maximal patterns reduce the reported set while preserving different information.

Closed vs. Maximal: A Quick Recap

Both summarize frequent patterns, but they preserve different information.

Dimension	Closed Frequent Itemset	Maximal Frequent Itemset
Definition	No proper superset has the same support	No proper superset is frequent
Information retained	Frequency and support information	Frequency boundary only
Compression	Moderate	Strong

**Every maximal frequent itemset is closed,
but a closed itemset need not be maximal.**

Equal support matters for closedness. Any frequent superset matters for maximality.

Recovering Support from Closed Patterns

- Closed patterns provide a **lossless compression** of frequent itemsets and their supports.

Transaction	Items
T1	{a, b, c}
T2	{a, b, c}
T3	{a, b}

minsup = 2

Closed pattern	Support
{a, b}	3
{a, b, c}	2

Only two closed patterns are stored

For any frequent itemset X :

$$\text{support}(X) = \max \{ \text{support}(Y) : Y \text{ is closed and } X \subseteq Y \}$$

Query {a}

$$\max \{ \text{support}(ab), \text{support}(abc) \} = \max \{ 3, 2 \} = 3$$

Query {c}

$$\text{support}(abc) = 2$$

The closure of X is the **largest superset** of X with the same support.

CLOSET+: Direct Mining of Closed Itemsets

□ Closure rule

$$\text{support}(X \cup \{i\}) = \text{support}(X)$$

Item i belongs to $\text{closure}(X)$

1. Project

Start with prefix X and its projected database



2. Merge and Prune

Merge same-support items into $\text{closure}(X)$
Skip redundant search branches



3. Output

Report $\text{closure}(X)$ only when no larger pattern has the same support

Choosing the Pattern Output

Output	<i>All Frequent</i>	<i>Closed</i>	<i>Maximal</i>
Retains	Every frequent pattern	Patterns and all supports	Frequency boundary
Use when	Complete enumeration is required	Support queries matter	A compact summary is enough

Note: output compression is a task decision, not only an implementation choice.

How to Evaluate if a Rule/Pattern Is Interesting?

- ❑ Pattern-mining will generate a large set of patterns/rules
 - ❑ Not all the generated patterns/rules are interesting
- ❑ Interestingness measures: **Objective** vs. **subjective**
 - ❑ **Objective** interestingness measures: Support, confidence, correlation, ...
 - ❑ **Subjective** interestingness measures: Different users may judge interestingness differently
 - ❑ Query-based: Relevant to a user's particular request
 - ❑ Judge against one's knowledge-base: unexpected, freshness, timeliness

Limitation of the Support-Confidence Framework

- Are s and c interesting in association rules: " $A \Rightarrow B$ " [s, c]? **Be careful!**
- Example: Suppose one school may have the following statistics on # of students who may play basketball and/or eat cereal: **2-way contingency table**

	play-basketball	not play-basketball	sum (row)
eat-cereal	400	350	750
not eat-cereal	200	50	250
sum(col.)	600	400	1000

- Association rule mining may generate the following:
 - $\text{play-basketball} \Rightarrow \text{eat-cereal}$ [$400/1000=40\%$, $400/600=66.7\%$] (higher s & c)
- But this strong association rule is misleading: The overall % of students eating cereal is $75\% > 66.7\%$, a more telling rule:
 - $\neg \text{play-basketball} \Rightarrow \text{eat-cereal}$ [$350/1000=35\%$, $350/400=87.5\%$] (high s & c)

Interestingness Measure: **Lift**

- Measure of dependent/correlated events: **lift**

$$lift(B, C) = \frac{c(B \rightarrow C)}{s(C)} = \frac{s(B \cup C)}{s(B) \times s(C)}$$

Lift is more telling than s & c

- Lift(B, C) may tell how B and C are correlated

- $lift(B, C) = 1$: B and C are independent
- > 1 : positively correlated
- < 1 : negatively correlated

	B	$\neg B$	Σ_{row}
C	400	350	750
$\neg C$	200	50	250
$\Sigma_{\text{col.}}$	600	400	1000

- For the example, $lift(B, C) = \frac{400/1000}{600/1000 \times 750/1000} = 0.89$
 $lift(B, \neg C) = \frac{200/1000}{600/1000 \times 250/1000} = 1.33$

- Thus, B and C are negatively correlated since $lift(B, C) < 1$;
B and $\neg C$ are positively correlated since $lift(B, \neg C) > 1$

Interestingness Measure: χ^2

- Another measure to test correlated events: χ^2

$$\chi^2 = \sum \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}}$$

- For the table on the right,

$$\chi^2 = \frac{(400 - 450)^2}{450} + \frac{(350 - 300)^2}{300} + \frac{(200 - 150)^2}{150} + \frac{(50 - 100)^2}{100} = 55.56$$

	B	$\neg B$	Σ_{row}
C	400 (450)	350 (300)	750
$\neg C$	200 (150)	50 (100)	250
Σ_{col}	600	400	1000

Expected value

Observed value

- Lookup χ^2 distribution table $\rightarrow$ B, C are correlated
- χ^2 -test shows B and C are negatively correlated since the expected value is 450 but the observed is only 400
- Thus, χ^2 is also more telling than the support-confidence framework

Lift and χ^2 : Are They Always Good Measures?

- ❑ **Null transactions:** Transactions that contain neither B nor C

- ❑ Let's examine the new dataset D



- ❑ BC (100) is much rarer than B¬C (1000) and ¬BC (1000), but there are many ¬B¬C (100000)

- ❑ Unlikely B & C will happen together!

- ❑ But, $\text{Lift}(B, C) = 8.44 \gg 1$ (Lift shows B and C are strongly positively correlated!)

- ❑ $\chi^2 = 670$: Observed(BC) $\gg$ expected value (11.85)

- ❑ Too many *null transactions* may “spoil the soup”!

	B	¬B	Σ_{row}
C	100	1000	1100
¬C	1000	100000	101000
$\Sigma_{\text{col.}}$	1100	101000	102100

null transactions

Contingency table with expected values added

	B	¬B	Σ_{row}
C	100 (11.85)	1000	1100
¬C	1000 (988.15)	100000	101000
$\Sigma_{\text{col.}}$	1100	101000	102100

Null Transactions in Sparse Data

- A null transaction contains neither item A nor item B.

	B	not B	Total
A	a	b	$a + b$
not A	c	d	$c + d$
Total	$a + c$	$b + d$	N

a A and B occur together

b A occurs without B

c B occurs without A

d Neither A nor B occurs

$$N = a + b + c + d$$

If only d changes, should the measured relationship between A and B change?

Null Invariance: Change d, Keep a, b, c Fixed

- The relationship evidence stays the same, but the number of null transactions changes.

Dataset	a	b	c	d	N
D ₁	20	10	10	60	100
D ₂	20	10	10	960	1,000

Measure	Formula	D ₁	D ₂	Null-invariant?
Jaccard	$a / (a + b + c)$	0.50	0.50	Yes
Kulczynski	$\frac{1}{2}[a/(a+b) + a/(a+c)]$	0.67	0.67	Yes
Lift	$aN / [(a+b)(a+c)]$	2.22	22.22	No

Null-invariant means: changing d alone does not change the measure value.

Which Measures Are Null-Invariant?

- A formula is **null-invariant** when it does not depend on d or N .

Measure	Formula	Null-invariant?	Main interpretation
Jaccard	$a / (a+b+c)$	Yes	Overlap among transactions containing A or B
Cosine	$a / \sqrt{[(a+b)(a+c)]}$	Yes	Normalized co-occurrence
Kulczynski	$\frac{1}{2}[a/(a+b)+a/(a+c)]$	Yes	Average of two directional confidences
Lift	$aN / [(a+b)(a+c)]$	No	Departure from independence in the full population
Chi-square	Observed vs. expected counts	No	Statistical test of dependence

**Null-invariant does not mean universally better.
Each measure answers a different question.**

Null Invariance Is Not Enough: Kulczynski and IR

- **Kulczynski measures symmetric association; IR (Imbalance Ratio) reveals support imbalance.**

$$\text{Kulc}(A,B) = \frac{1}{2}[p + q]$$

$$p = P(B|A), \quad q = P(A|B)$$

$$\text{IR}(A,B) = |s(A) - s(B)| / [s(A) + s(B) - s(A \cap B)]$$

IR = 0: balanced supports
IR near 1: strong imbalance

Example	a	b	c	p	q	Kulc	IR
Balanced pair	20	10	10	0.67	0.67	0.67	0.00
Imbalanced pair	10	0	20	1.00	0.33	0.67	0.67

Both examples have Kulc = 0.67, but the second pair is strongly directional and imbalanced.

Note: report IR alongside Kulczynski when item supports differ substantially.

Choosing Measures for Pattern Evaluation

- Choose the measure that matches the analytical question and the meaning of null cases.

Question	Suggested measure	Why
How much do the active cases overlap?	Jaccard	Ignores neither-neither cases
How strong is normalized co-occurrence?	Cosine	Adjusts for different marginal counts
How strong is the association in both directions?	Kulczynski + IR	Combines association strength with imbalance
How far is the full population from independence?	Lift	Uses marginal probabilities and N
Is the dependence statistically significant?	Chi-square	Uses expected counts and sample size

Sparse data; unstable null population

Prefer a null-invariant measure

Stable, meaningful null population

Lift or chi-square may be appropriate

Quick Check

Question: A rule $X \rightarrow Y$ has 90% confidence but lift ≈ 1 .
What is the best interpretation?

A	B	C	D
X strongly causes Y	Y is common and X adds little information	The rule must be rare	Apriori counted support incorrectly

Answer: B.

lift = confidence / $s(Y) \approx 1$. Therefore, confidence = $P(Y|X) \approx s(Y) = P(Y)$. Observing X does not substantially change the probability of Y. Confidence must be interpreted relative to the consequent's base rate.

Part 2 Summary

Frequent-pattern mining requires separate decisions about computation, output, and interpretation.

1 Efficient search

- ❑ Apriori and DHP generate candidates level by level and prune them early.
- ❑ ECLAT performs depth-first search using TID-set intersections.
- ❑ FP-growth mines conditional FP-trees without explicit candidate generation.

2 Compact output

- ❑ Closed itemsets remove same-support redundancy while preserving support information.
- ❑ Maximal itemsets provide stronger compression but discard exact subset supports.

3 Meaningful evaluation

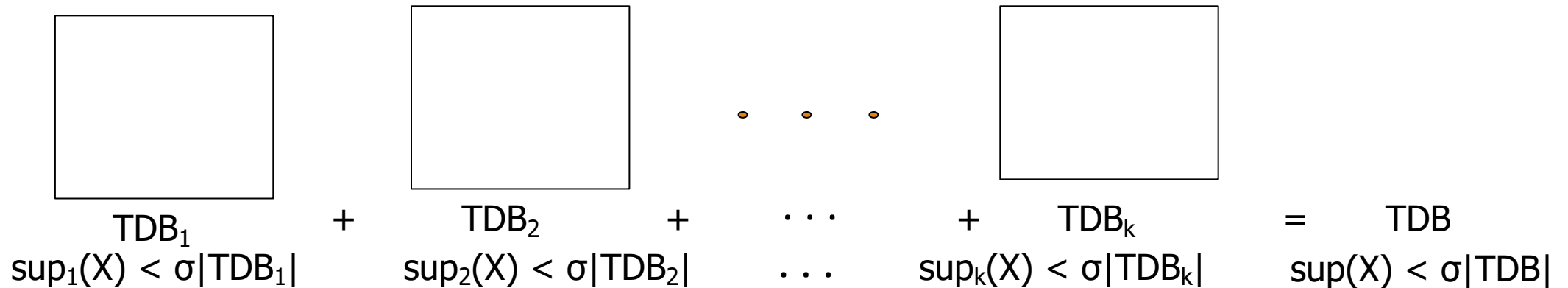
- ❑ Support and confidence describe frequency, but they do not establish dependence.
- ❑ Lift compares confidence with the consequent's base rate.
- ❑ When null transactions are unstable, use null-invariant measures.

Appendix

Backup slides for Part 2

Partitioning: Scan Database Only Twice

- **Theorem:** *Any itemset that is potentially frequent in TDB must be frequent in at least one of the partitions of TDB*



- **Method: Scan DB twice** (A. Savasere, E. Omiecinski and S. Navathe, VLDB'95)
 - **Scan 1:** Partition database so that each partition can fit in main memory **(why?)**
 - Mine local frequent patterns in this partition
 - **Scan 2:** Consolidate global frequent patterns
 - Find global frequent itemset candidates (those frequent in at least one partition)
 - Find the true frequency of those candidates, by scanning TDBi one more time

Direct Hashing and Pruning (DHP)

- **Hashing:** $v = \text{hash}(\text{itemset})$
- **1st scan:** When counting the 1-itemset, hash 2-itemset to calculate the bucket count
- Example: At the 1st scan of TDB, count 1-itemset, and hash 2-itemsets in the transaction to its **bucket**
 - Bucket V_1 : {ab, ad, ce}
 - Bucket V_2 : {bd, be, de}
 - Bucket V_n : ...
- At the end of the first scan,
 - if $\text{minsup} = 80$, prune ab, ad, ce, because their shared bucket, {ab, ad, ce}, count is < 80

V might be same for different itemset

Itemsets	Count
{ab, ad, ce}	35
{bd, be, de}	298
.....	...
{yz, qs, wt}	58

Hash Table



Check the minsup

A key observation: A k -itemset cannot be frequent if its corresponding hashing bucket count is below the *minsup* threshold

A low bucket count guarantees infrequency; a high bucket count only indicates possibility.

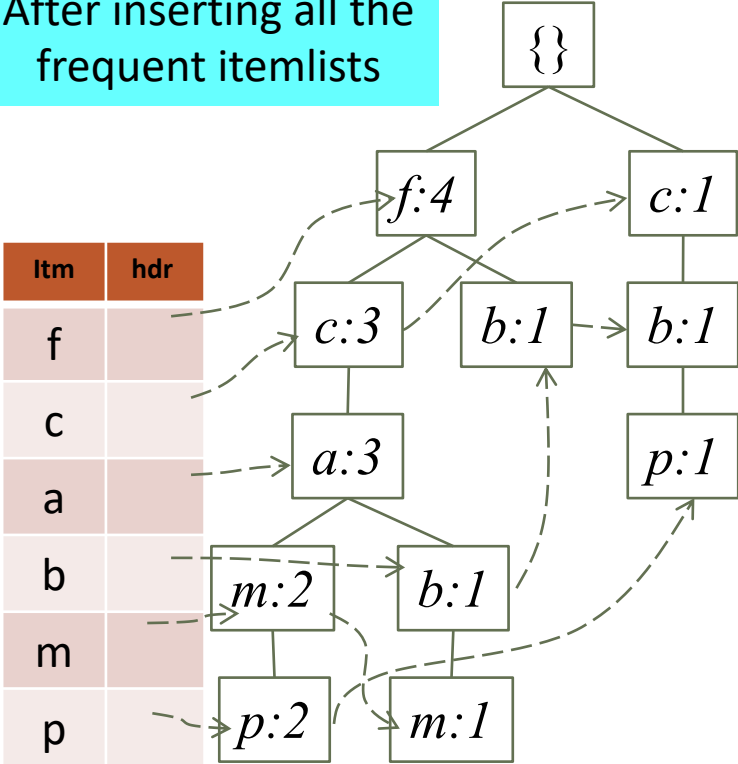
Additional Apriori Variants

Method	Core idea
Dynamic itemset counting	Introduce candidates at checkpoints within a database scan
Sampling	Mine a sample with a relaxed threshold, then verify candidates on the full database
Support lower bounding	Use safe bounds to eliminate candidates before exact counting
Tree projection	Project transactions into search-specific trees
H-Miner	Use hyperlinked arrays to avoid repeated projected-database construction
LCM / hypercube decomposition	Enumerate frequent or closed itemsets with specialized closure and prefix techniques

These variants reduce scans, candidates, or representation cost in different ways.

FP-Tree Construction by Prefix Merging

After inserting all the frequent itemlists



Transaction inserted	Effect on the tree
f, c, a, m, p	Create path f:1, c:1, a:1, m:1, p:1
f, c, a, b, m	Increment f-c-a to 2, then branch to b-m
f, b	Increment f, then add a second b node
c, b, p	Create a new root branch beginning with c
f, c, a, m, p	Increment the complete first path

The tree merges prefixes; it does not merge identical items that occur under different prefixes.

Formula Reference

Quantity	Formula
Support count	$\sigma(X)$ = number of transactions containing X
Relative support	$s(X) = \sigma(X) / N$
Confidence	$\text{conf}(X \Rightarrow Y) = s(X \cup Y) / s(X)$
Lift	$\text{lift}(X \Rightarrow Y) = \text{conf}(X \Rightarrow Y) / s(Y)$
Expected count	E_{ij} = row total $\times$ column total / grand total
Chi-square	$\chi^2 = \sum (O - E)^2 / E$, $df = (r - 1)(c - 1)$
Kulczynski	$\text{Kulc}(A, B) = \frac{1}{2}[P(B A) + P(A B)]$

State the denominator before interpreting a probability.

Lift: Worked Example

A contingency table shows how two categorical variables occur together across all combinations.

Member type	Short	Not short	Row total
Subscriber	50	10	60
Casual	30	10	40
Column total	80	20	100

Rows describe member type; columns describe duration class.

Lift: Worked Example

$$\text{lift}(\text{Subscriber} \Rightarrow \text{Short}) \approx 1.04$$

Lift value	Interpretation
> 1	Positive association relative to independence
= 1	Independence baseline
< 1	Negative association relative to independence

Lift measures multiplicative departure from independence, not causal effect.

Chi-Square, Degrees of Freedom, and p-Value

$$\chi^2 = \sum (O - E)^2 / E$$

$$df = (r - 1)(c - 1) = 1$$

Cell	O	E	(O-E) ² /E
Subscriber, Short	50	48	0.083
Subscriber, Not short	10	12	0.333
Casual, Short	30	32	0.125
Casual, Not short	10	8	0.500
Total			$\chi^2 \approx 1.04$

Right-tail area under
 $\chi^2(df=1)$
beyond 1.04

$p \approx 0.31$

No strong evidence against
independence

The p-value is not the probability that the null hypothesis is true.

Interestingness Measures & Null-Invariance

- *Null invariance* means: The number of null transactions **does not matter** (*Does not change the measure value*).
- A few interestingness measures: Some are null invariant

Measure	Definition
$\chi^2(A, B)$	$\sum_{i,j} \frac{(e(a_i, b_j) - o(a_i, b_j))^2}{e(a_i, b_j)}$
$Lift(A, B)$	$\frac{s(A \cup B)}{s(A) \times s(B)}$
$Allconf(A, B)$	$\frac{s(A \cup B)}{\max\{s(A), s(B)\}}$
$\tau(A, B)$	$\frac{s(A \cap B)}{s(A \cup B)}$

Let

$$p = \frac{s(A \cup B)}{s(A)} = P(B|A)$$

$$q = \frac{s(A \cup B)}{s(B)} = P(A|B)$$

p, q are null invariant

Essentially min, max,
mean variants of p, q

Null Invariance: An Important Property

□ Why is null invariance crucial for the analysis of massive transaction data?

▣ Many transactions may contain neither milk nor coffee!

milk vs. coffee contingency table

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

□ Lift and χ^2 are not null-invariant: their values can change when many neither-neither transactions are added.

□ Many association measures are not null-invariant.

Null-transactions
w.r.t. *m* and *c*

Data set	<i>mc</i>	$\neg mc$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	χ^2	<i>Lift</i>
<i>D</i> ₁	10,000	1,000	1,000	100,000	90557	9.26
<i>D</i> ₂	10,000	1,000	1,000	100	0	1
<i>D</i> ₃	100	1,000	1,000	100,000	670	8.44
<i>D</i> ₄	1,000	1,000	1,000	100,000	24740	25.75
<i>D</i> ₅	1,000	100	10,000	100,000	8173	9.18
<i>D</i> ₆	1,000	10	100,000	100,000	965	1.97

Comparison of Null-Invariant Measures

- ❑ Not all null-invariant measures are created equal
- ❑ Which one is better?
 - ❑ D4—D6 differentiate the null-invariant measures
 - ❑ Kulc (Kulczynski 1927) holds firm and is in balance of both directional implications

2-variable contingency table

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	<i>m</i> $\neg c$	$\neg m \neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

All 5 are null-invariant

Data set	<i>mc</i>	$\neg mc$	<i>m</i> $\neg c$	$\neg m \neg c$	<i>AllConf</i>	Jaccard	<i>Cosine</i>	<i>Kulc</i>	<i>MaxConf</i>
<i>D</i> ₁	10,000	1,000	1,000	100,000	0.91	0.83	0.91	0.91	0.91
<i>D</i> ₂	10,000	1,000	1,000	100	0.91	0.83	0.91	0.91	0.91
<i>D</i> ₃	100	1,000	1,000	100,000	0.09	0.05	0.09	0.09	0.09
<i>D</i> ₄	1,000	1,000	1,000	100,000	0.5	0.33	0.5	0.5	0.5
<i>D</i> ₅	1,000	100	10,000	100,000	0.09	0.09	0.29	0.5	0.91
<i>D</i> ₆	1,000	10	100,000	100,000	0.01	0.01	0.10	0.5	0.99

Subtle: They disagree on those cases

Null-Invariant Measure Family

Measure	Formula	Interpretation
Jaccard	$a / (a+b+c)$	Shared positives among all non-null transactions
Cosine	$a / \sqrt{((a+b)(a+c))}$	Geometric balance of the two supports
All-confidence	$\min[a/(a+b), a/(a+c)]$	Weaker directional confidence
Max-confidence	$\max[a/(a+b), a/(a+c)]$	Stronger directional confidence
Kulczynski	$\frac{1}{2}[a/(a+b) + a/(a+c)]$	Average of both directions
IR	$ b-c / (a+b+c)$	Marginal support imbalance

Null invariance means the neither-neither count d does not change the measure.

Imbalance Ratio with Kulczynski Measure

- IR (Imbalance Ratio): measure the imbalance of two itemsets A and B in rule implications:

$$IR(A, B) = \frac{|s(A) - s(B)|}{s(A) + s(B) - s(A \cup B)}$$

- Kulczynski and Imbalance Ratio (IR) together present a clear picture for all the three datasets D4 through D6
 - D4 is neutral & balanced; D5 is neutral but imbalanced
 - D6 is neutral but very imbalanced

Data set	mc	$\neg mc$	$m\neg c$	$\neg m\neg c$	Jaccard	Cosine	Kulc	IR
D_1	10,000	1,000	1,000	100,000	0.83	0.91	0.91	0
D_2	10,000	1,000	1,000	100	0.83	0.91	0.91	0
D_3	100	1,000	1,000	100,000	0.05	0.09	0.09	0
D_4	1,000	1,000	1,000	100,000	0.33	0.5	0.5	0
D_5	1,000	100	10,000	100,000	0.09	0.29	0.5	0.89
D_6	1,000	10	100,000	100,000	0.01	0.10	0.5	0.99

Example: Analysis of DBLP Coauthor Relationships

- DBLP: Computer science research publication bibliographic database
 - > 3.8 million entries on authors, paper, venue, year, and other information

ID	Author <i>A</i>	Author <i>B</i>	$s(A \cup B)$	$s(A)$	$s(B)$	Jaccard	Cosine	Kulc
1	Hans-Peter Kriegel	Martin Ester	28	146	54	0.163 (2)	0.315 (7)	0.355 (9)
2	Michael Carey	Miron Livny	26	104	58	0.191 (1)	0.335 (4)	0.349 (10)
3	Hans-Peter Kriegel	Joerg Sander	24	146	36	0.152 (3)	0.331 (5)	0.416 (8)
4	Christos Faloutsos	Spiros Papadimitriou	20	162	26	0.119 (7)	0.308 (10)	0.446 (7)
5	Hans-Peter Kriegel	Martin Pfeifle	18	146	18	0.123 (6)	0.351 (2)	0.562 (2)
6	Hector Garcia-Molina	Wilburt Labio	16	144	18	0.110 (9)	0.314 (8)	0.500 (4)
7	Divyakant Agrawal	Wang Hsiung	16	120	16	0.133 (5)	0.365 (1)	0.567 (1)
8	Elke Rundensteiner	Murali Mani	16	104	20	0.148 (4)	0.351 (3)	0.477 (6)
9	Divyakant Agrawal	Oliver Po	12	120	12	0.100 (10)	0.316 (6)	0.550 (3)
10	Gerhard Weikum	Martin Theobald	12	106	14	0.111 (8)	0.312 (9)	0.485 (5)

Advisor-advisee relation: Kulc: high, Jaccard: low, cosine: middle

- Which pairs of authors are strongly related? Is A the advisor, or the advisee?
 - Use Kulc to find Advisor-advisee, close collaborators

Choosing Measures for Pattern Evaluation

- ❑ Null transactions are common in sparse data: most baskets may contain neither item.
- ❑ Use lift and chi-square when neither-neither cases are meaningful and not overwhelming.
- ❑ Use null-invariant measures when the null count mainly reflects sampling.
- ❑ Use Kulczynski with IR to separate association strength from directional imbalance.

Application: In DBLP, mine frequent collaborator groups and examine possible advisor-advisee directions over time.

Recommended Readings (Efficient Pattern Mining Methods)

- ❑ R. Agrawal and R. Srikant, “Fast algorithms for mining association rules”, VLDB'94
- ❑ A. Savasere, E. Omiecinski, and S. Navathe, “An efficient algorithm for mining association rules in large databases”, VLDB'95
- ❑ J. S. Park, M. S. Chen, and P. S. Yu, “An effective hash-based algorithm for mining association rules”, SIGMOD'95
- ❑ S. Sarawagi, S. Thomas, and R. Agrawal, “Integrating association rule mining with relational database systems: Alternatives and implications”, SIGMOD'98
- ❑ M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, “Parallel algorithm for discovery of association rules”, Data Mining and Knowledge Discovery, 1997
- ❑ J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation”, SIGMOD'00
- ❑ M. J. Zaki and Hsiao, “CHARM: An Efficient Algorithm for Closed Itemset Mining”, SDM'02
- ❑ J. Wang, J. Han, and J. Pei, “CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets”, KDD'03
- ❑ C. C. Aggarwal, M.A., Bhuiyan, M. A. Hasan, “Frequent Pattern Mining Algorithms: A Survey”, in Aggarwal and Han (eds.): Frequent Pattern Mining, Springer, 2014

Recommended Readings (Pattern Evaluation)

- ❑ C. C. Aggarwal and P. S. Yu. A New Framework for Itemset Generation. PODS'98
- ❑ S. Brin, R. Motwani, and C. Silverstein. Beyond market basket: Generalizing association rules to correlations. SIGMOD'97
- ❑ M. Klemettinen, H. Mannila, P. Ronkainen, H. Toivonen, and A. I. Verkamo. Finding interesting rules from large sets of discovered association rules. CIKM'94
- ❑ E. Omiecinski. Alternative Interest Measures for Mining Associations. TKDE'03
- ❑ P.-N. Tan, V. Kumar, and J. Srivastava. Selecting the Right Interestingness Measure for Association Patterns. KDD'02
- ❑ T. Wu, Y. Chen and J. Han, Re-Examination of Interestingness Measures in Pattern Mining: A Unified Framework, Data Mining and Knowledge Discovery, 21(3):371-397, 2010